МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Нижегородский государственный технический университет
им. Р.Е. Алексеева»

Арзамасский политехнический институт (филиал)

**ОТЧЕТ**

о выполнении практической работы № 5
по дисциплине «Основы тестирования программного обеспечения»

Тема: «Нагрузочное тестирование REST-сервиса на FastAPI средствами Apache JMeter»

Выполнил: студент гр. АЗИС 23-1
Рубцов М.И.

Проверил: преподаватель
Комаров А.О.

Арзамас, 2026

## 1 Цель работы

Целью работы является освоение инструмента Apache JMeter для тестирования производительности веб-приложений: развернуть тестовый REST-сервис на FastAPI, создать план нагрузочного теста с группой потоков, HTTP-запросами и слушателями, выполнить тест и проанализировать результаты (времена отклика, пропускную способность, процент ошибок).

## 2 Задание

В соответствии с методическими указаниями требуется: установить зависимости (`fastapi[all]`, `uvicorn`), создать файл `main.py` с сервером «аукциона» (таблица `items` в SQLite, методы `POST /items/`, `GET /items/`, `GET /items/{item_id}`) и запустить его командой `uvicorn main:app --reload`; в JMeter создать тестовый план с Thread Group (5 пользователей, разгон 10 с, 1 итерация), HTTP Request «Create Item» (POST /items/ с JSON-телом) и «Get Items» (GET /items/), добавить слушатель View Results Tree, запустить тест и просмотреть результаты.

## 3 Теоретические сведения

Apache JMeter – инструмент с открытым исходным кодом для нагрузочного и функционального тестирования веб-приложений. Он имитирует множество пользователей, отправляющих запросы к серверу, измеряет время отклика и выявляет узкие места. Основные элементы плана: Thread Group задает количество виртуальных пользователей, период разгона и число итераций; Samplers (HTTP Request и др.) отправляют запросы; Config Elements (HTTP Header Manager, CSV Data Set Config) настраивают запросы; Assertions проверяют ответы; Listeners (View Results Tree, Summary Report, Aggregate Report) собирают и отображают результаты. Тест можно запускать в графическом режиме (для отладки) и в режиме командной строки `-n` с записью результатов в файл `.jtl` и генерацией HTML-отчета – так рекомендуется проводить реальные нагрузочные испытания.

Ключевые метрики: среднее, минимальное и максимальное время отклика, медиана и перцентили (90 %, 95 %, 99 %), пропускная способность (запросов в

секунду) и доля ошибок. Индекс APDEX в HTML-отчете агрегирует удовлетворенность пользователей по порогам времени отклика.

## 4 Развертывание тестируемого сервера

Сервер реализован в файле `main.py` по тексту методических указаний без изменений (листинг 1). Окружение: Python 3.11.15, FastAPI 0.141.1, Uvicorn 0.46.0, Pydantic 2.13.3. Сервер запущен командой `uvicorn main:app --reload --port 8000`; фрагмент его журнала показан на рисунке 1.

Листинг 1 – main.py – тестируемый сервер

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from typing import List
import sqlite3


app = FastAPI()


# Database setup
def init_db():
    conn = sqlite3.connect('auction.db')
    cursor = conn.cursor()
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS items (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT NOT NULL,
            description TEXT,
            price REAL NOT NULL
        )
    ''')
    conn.commit()
    conn.close()


init_db()


class Item(BaseModel):
    name: str
    description: str = None
    price: float


@app.post("/items/", response_model=Item)
def create_item(item: Item):
    conn = sqlite3.connect('auction.db')
    cursor = conn.cursor()
    cursor.execute('''
        INSERT INTO items (name, description, price)
        VALUES (?, ?, ?)
    ''', (item.name, item.description, item.price))
    conn.commit()
    conn.close()
    return item


@app.get("/items/", response_model=List[Item])
def get_items():
    conn = sqlite3.connect('auction.db')
    cursor = conn.cursor()
    cursor.execute('SELECT * FROM items')
    rows = cursor.fetchall()
    conn.close()
```

```
    return [Item(id=row[0], name=row[1], description=row[2], price=row[3]) for row in
rows]


@app.get("/items/{item_id}", response_model=Item)
def get_item(item_id: int):
    conn = sqlite3.connect('auction.db')
    cursor = conn.cursor()
    cursor.execute('SELECT * FROM items WHERE id = ?', (item_id,))
    row = cursor.fetchone()
    conn.close()
    if row is None:
        raise HTTPException(status_code=404, detail="Item not found")
    return Item(id=row[0], name=row[1], description=row[2], price=row[3])
```

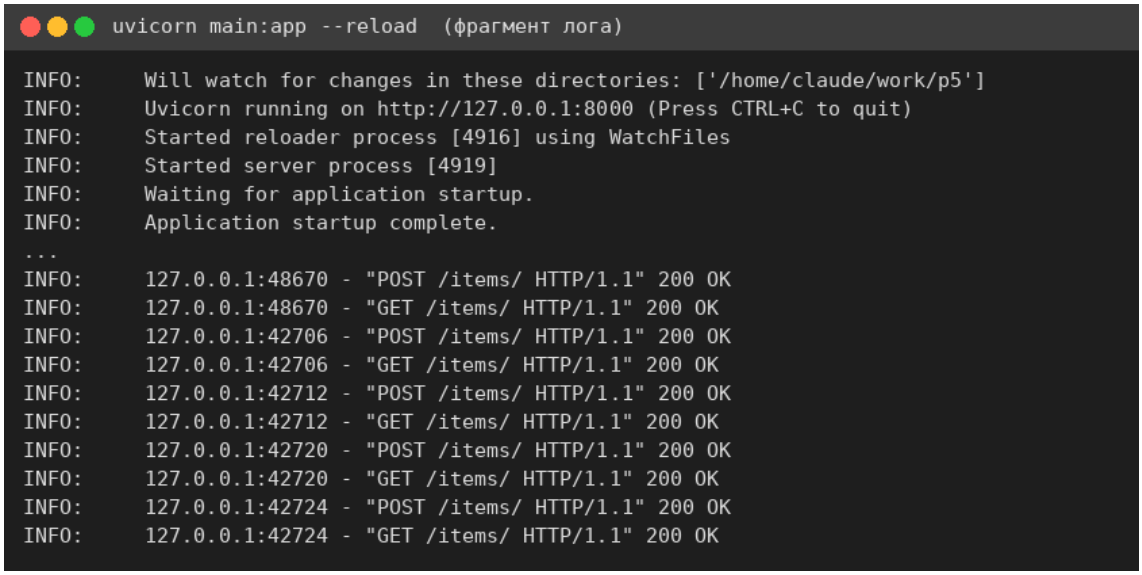



Рисунок 1 – Журнал uvicorn: запуск сервера и запросы JMeter

Работоспособность API проверена утилитой curl (рисунок 2): создание элемента возвращает 200 OK и JSON с полями name, description, price; список элементов и элемент по идентификатору возвращаются с кодом 200; запрос несуществующего идентификатора – 404 `{"detail":"Item not found"}`. Автоматически сгенерированная документация Swagger UI доступна по адресу /docs (рисунок 3).

curl — проверка API

```
$ curl -s -X POST http://127.0.0.1:8000/items/ -H "Content-Type: application/json" -d '{"name": "Item 1", "des
cription": "A description of Item 1", "price": 100.0}'
HTTP/1.1 200 OK
date: Sat, 12 Sep 2026 09:57:41 GMT
server: uvicorn
content-length: 71
content-type: application/json

{"name":"Item 1","description":"A description of Item 1","price":100.0}

$ curl -s http://127.0.0.1:8000/items/
HTTP/1.1 200 OK
date: Sat, 12 Sep 2026 09:57:41 GMT
server: uvicorn
content-length: 73
content-type: application/json

[{"name":"Item 1","description":"A description of Item 1","price":100.0}]

$ curl -s http://127.0.0.1:8000/items/1
HTTP/1.1 200 OK
date: Sat, 12 Sep 2026 09:57:41 GMT
server: uvicorn
content-length: 71
content-type: application/json

{"name":"Item 1","description":"A description of Item 1","price":100.0}

$ curl -s http://127.0.0.1:8000/items/999
HTTP/1.1 404 Not Found
date: Sat, 12 Sep 2026 09:57:41 GMT
server: uvicorn
content-length: 27
content-type: application/json

{"detail":"Item not found"}
```

Рисунок 2 – Проверка API утилитой curl

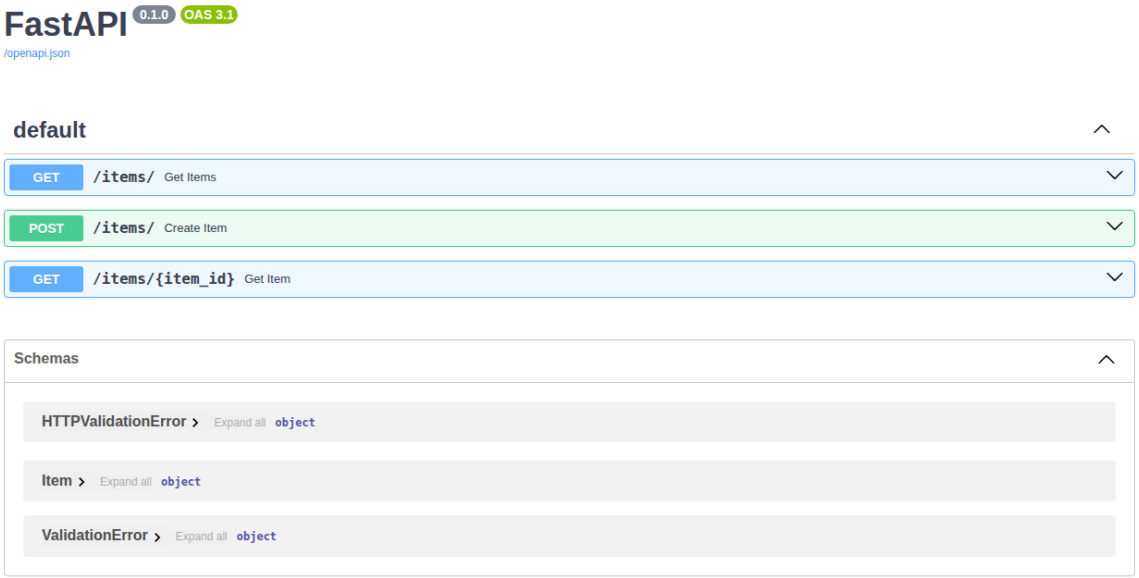



Рисунок 3 – Swagger UI сервера: три метода и схема Item

При проверке обнаружен дефект сервера: модель `Item` не содержит поля `id`, хотя в `get_items` и `get_item` создается `Item(id=row[0], ...)`. Pydantic 2 по умолчанию игнорирует лишние поля, поэтому сервер не падает, но клиенты не получают идентификаторов записей – в ответе `GET /items/` все элементы неразличимы. Также аннотация `description: str = None` некорректна (должно быть `Optional[str]`), а `POST /items/` возвращает входные данные, а не созданную запись. Код сервера, согласно заданию, оставлен без изменений; дефекты зафиксированы в разделе 7.

## 5 Создание тестового плана в JMeter

Использован Apache JMeter 5.5 (OpenJDK 21). Официальные зеркала загрузки Apache были недоступны из учебного окружения, поэтому дистрибутив 5.5 получен из публичного репозитория; функциональность версии для данного задания эквивалентна 5.6. GUI запускался на виртуальном X-сервере (Xvfb + Openbox), все снимки – реальные окна JMeter. Структура плана `auction_test.jmx` приведена в таблице 1; к элементам методических указаний добавлены HTTP Header Manager (без заголовка `Content-Type: application/json` FastAPI отвечал бы 422) и два Response Assertion на код 200, а также слушатели Summary Report и Aggregate Report для количественного анализа.

Таблица 1 – Структура тестового плана auction_test.jmx

| Элемент | Тип | Параметры |
|---|---|---|
| Auction API Test | Test Plan | – |
| Thread Group | Threads (Users) | Number of Threads = 5; Ramp-Up Period = 10 s; Loop Count = 1 |
| Create Item | HTTP Request | 127.0.0.1:8000, POST, /items/, Body Data: {"name": "Item 1", "description": "A description of Item 1", "price": 100.0} |
| HTTP Header Manager | Config Element | Content-Type: application/json (дочерний элемент Create Item) |
| Response Assertion | Assertion | Response Code equals 200 (для Create Item и Get Items) |

| Элемент | Тип | Параметры |
|---|---|---|
| Get Items | HTTP Request | 127.0.0.1:8000, GET, /items/ |
| View Results Tree | Listener | просмотр каждого запроса и ответа |
| Summary Report, Aggregate Report | Listener | сводные метрики |

Дерево плана и настройки Thread Group показаны на рисунке 4: 5 потоков, разгон 10 с, одна итерация. Настройки сэмплера «Create Item» с телом запроса в формате JSON – на рисунке 5, сэмплера «Get Items» – на рисунке 6, менеджера заголовков и проверки кода ответа – на рисунках 7 и 8.

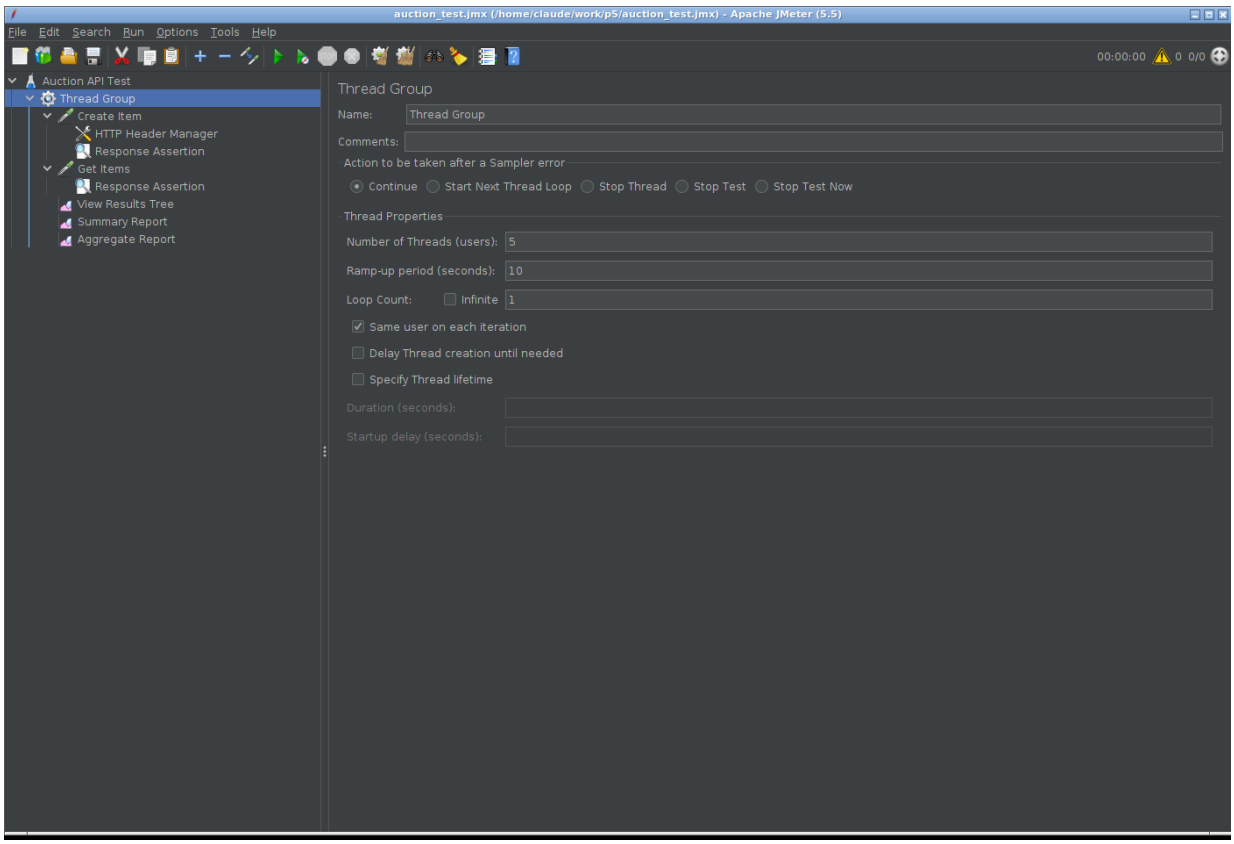


Рисунок 4 – Тестовый план и параметры Thread Group (5 / 10 / 1)

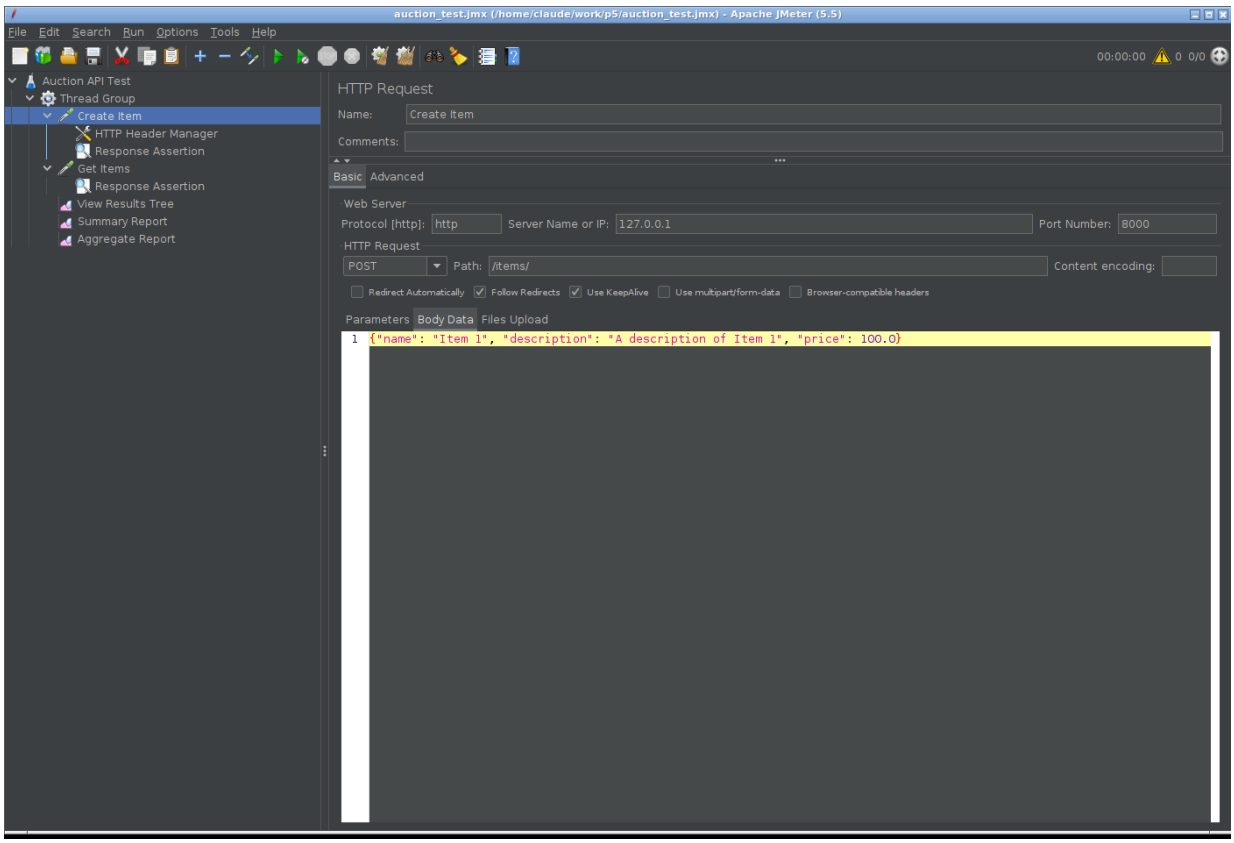



Рисунок 5 – HTTP Request «Create Item»: POST /items/ с Body Data

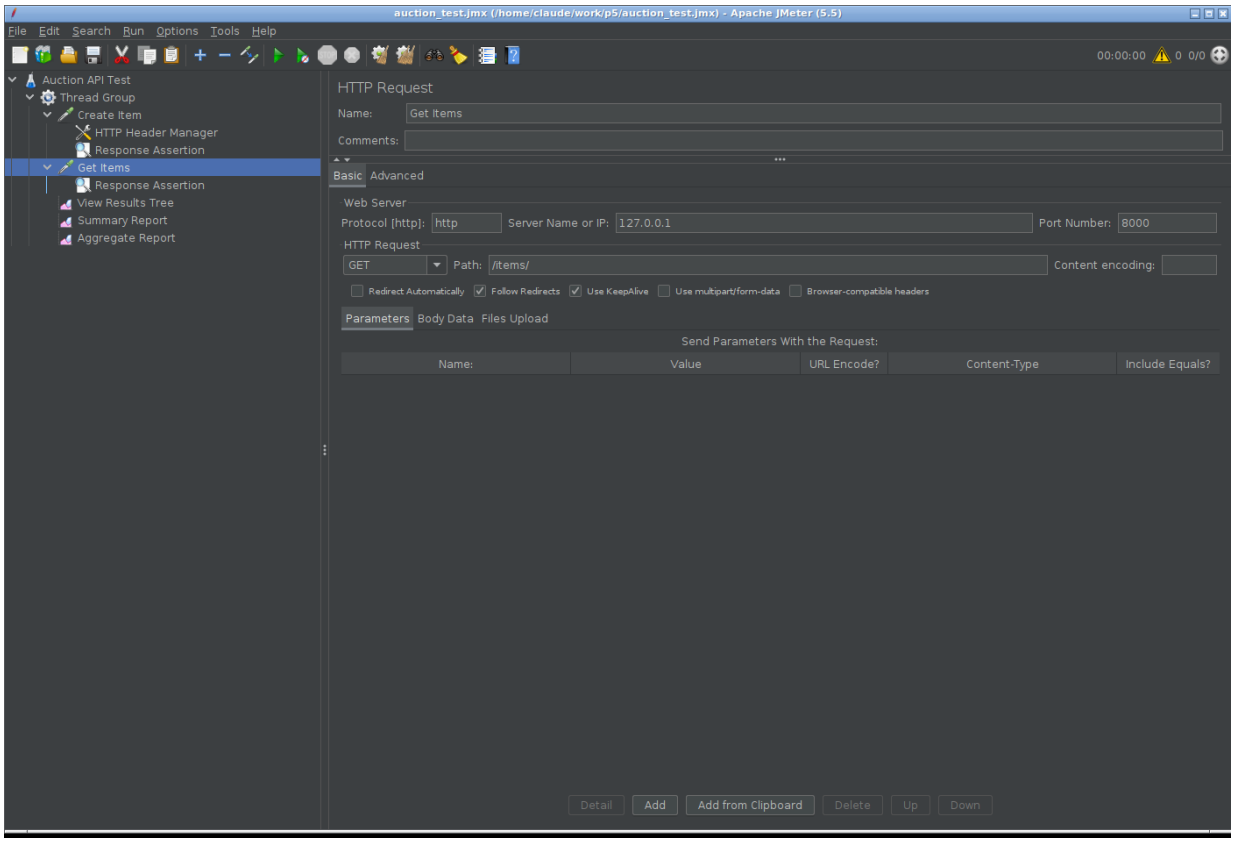



Рисунок 6 – HTTP Request «Get Items»: GET /items/

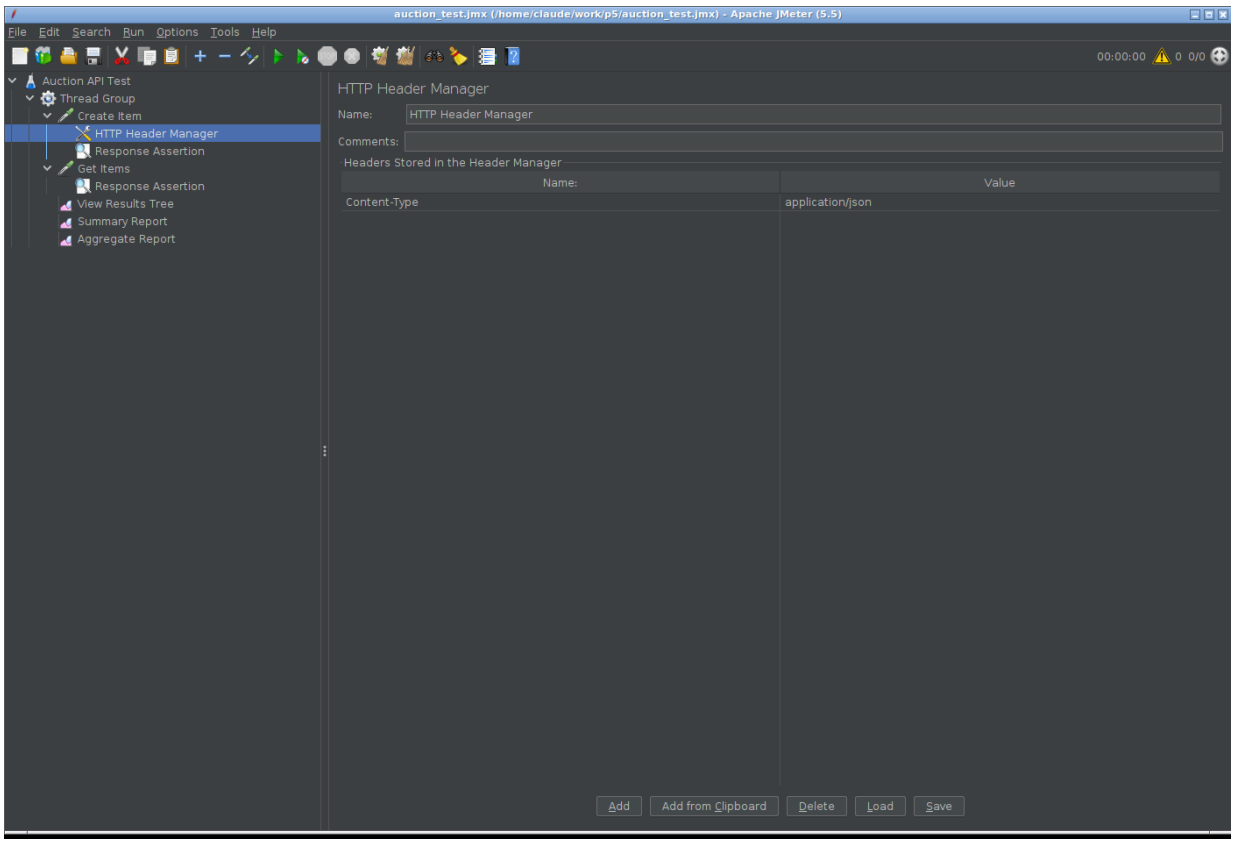



Рисунок 7 – HTTP Header Manager: Content-Type = application/json

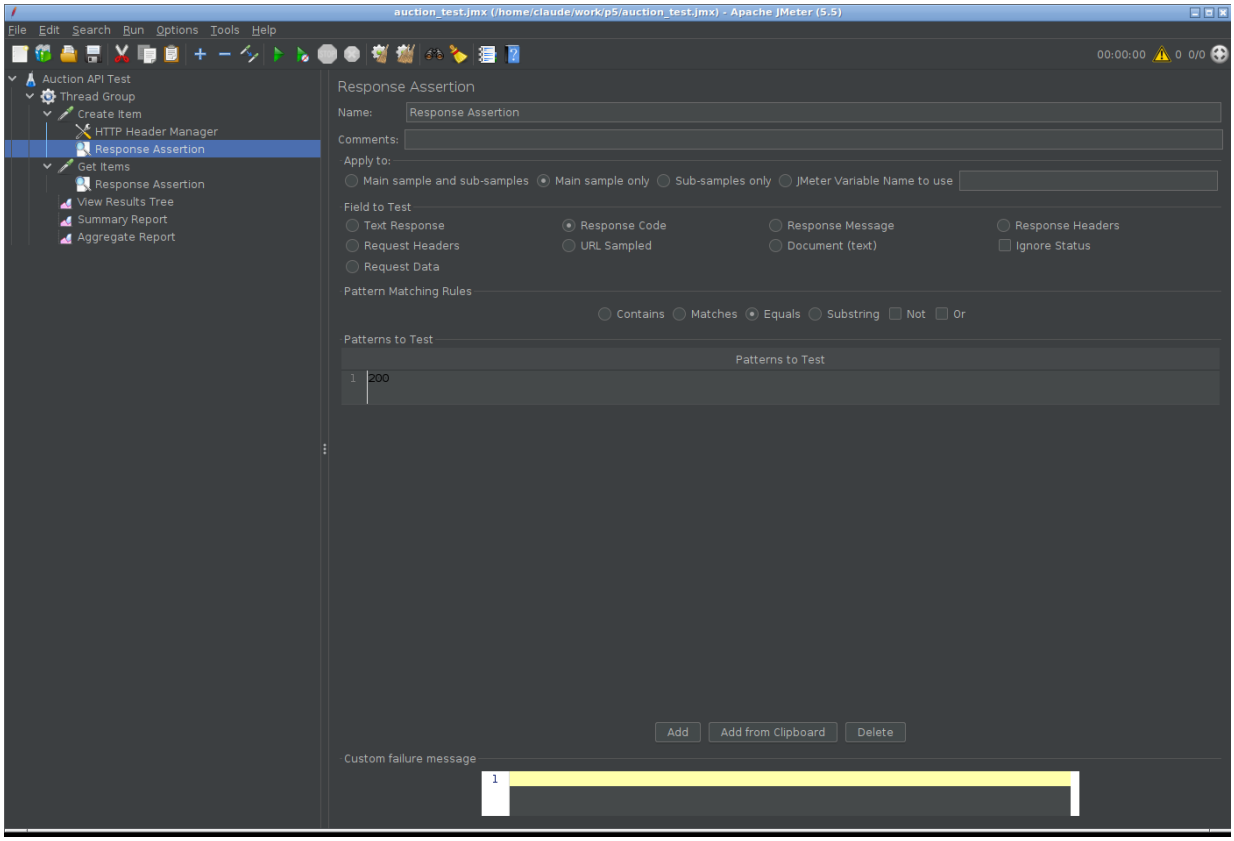



Рисунок 8 – Response Assertion: код ответа равен 200

# 6 Выполнение теста и результаты

## 6.1 Запуск в графическом режиме

Тест запущен кнопкой Start (Ctrl+R). В View Results Tree (рисунок 9) все десять сэмплов – пять «Create Item» и пять «Get Items» – отмечены зеленым: коды ответов 200, утверждения пройдены. Для первого запроса «Create Item» открыта вкладка Response data: тело ответа `{"name":"Item 1","description":"A description of Item 1","price":100.0}`; на рисунке 10 показана вкладка Request с отправленным JSON.

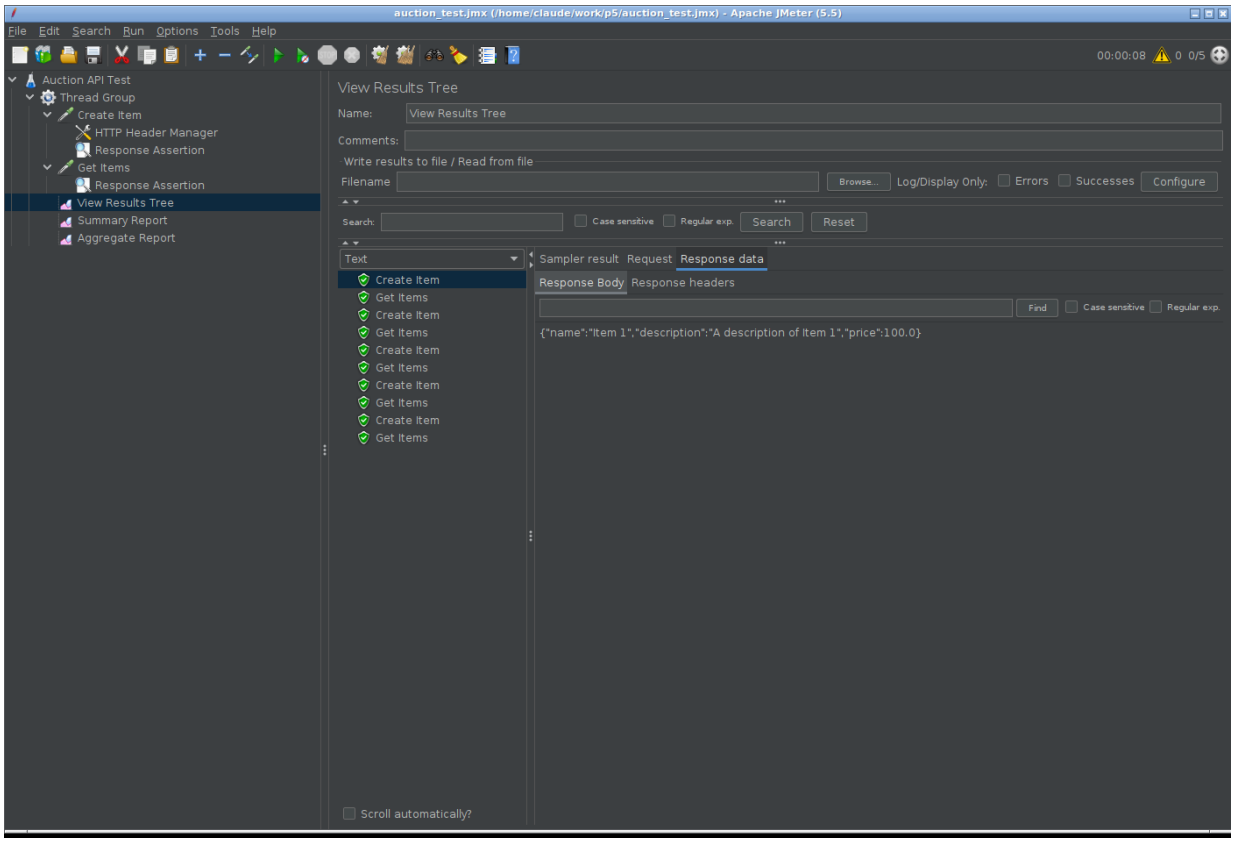



Рисунок 9 – View Results Tree: 10 успешных результатов, тело ответа POST /items/

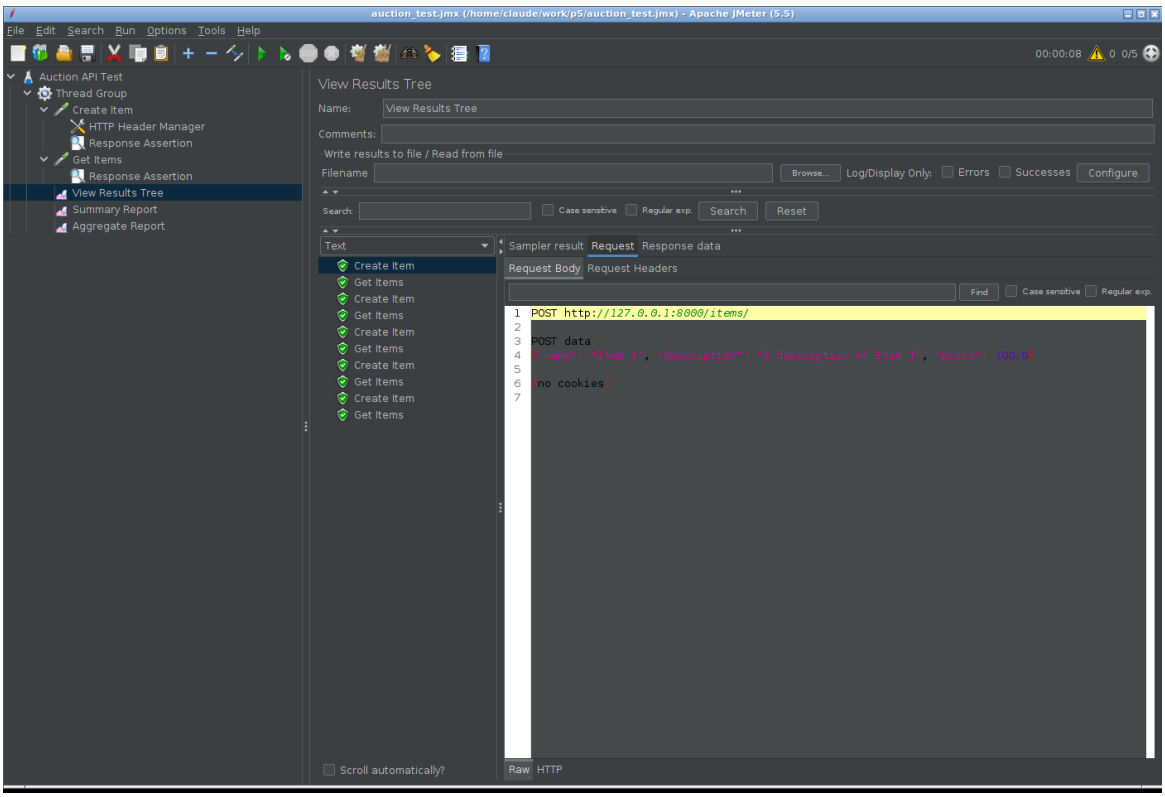



Рисунок 10 – View Results Tree: отправленный запрос Create Item

Сводные метрики Summary Report и Aggregate Report показаны на рисунках 11 и 12, числовые значения сведены в таблицу 2.

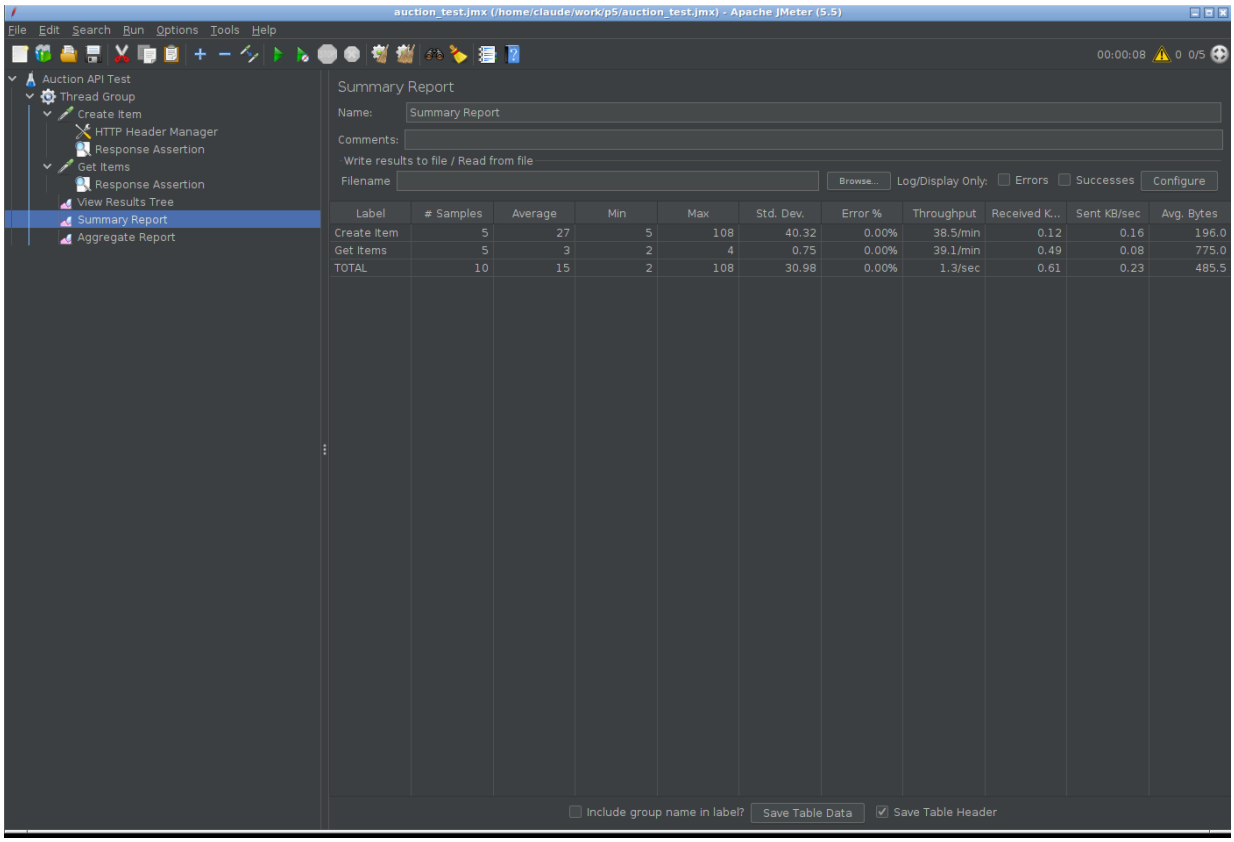



Рисунок 11 – Summary Report после выполнения теста

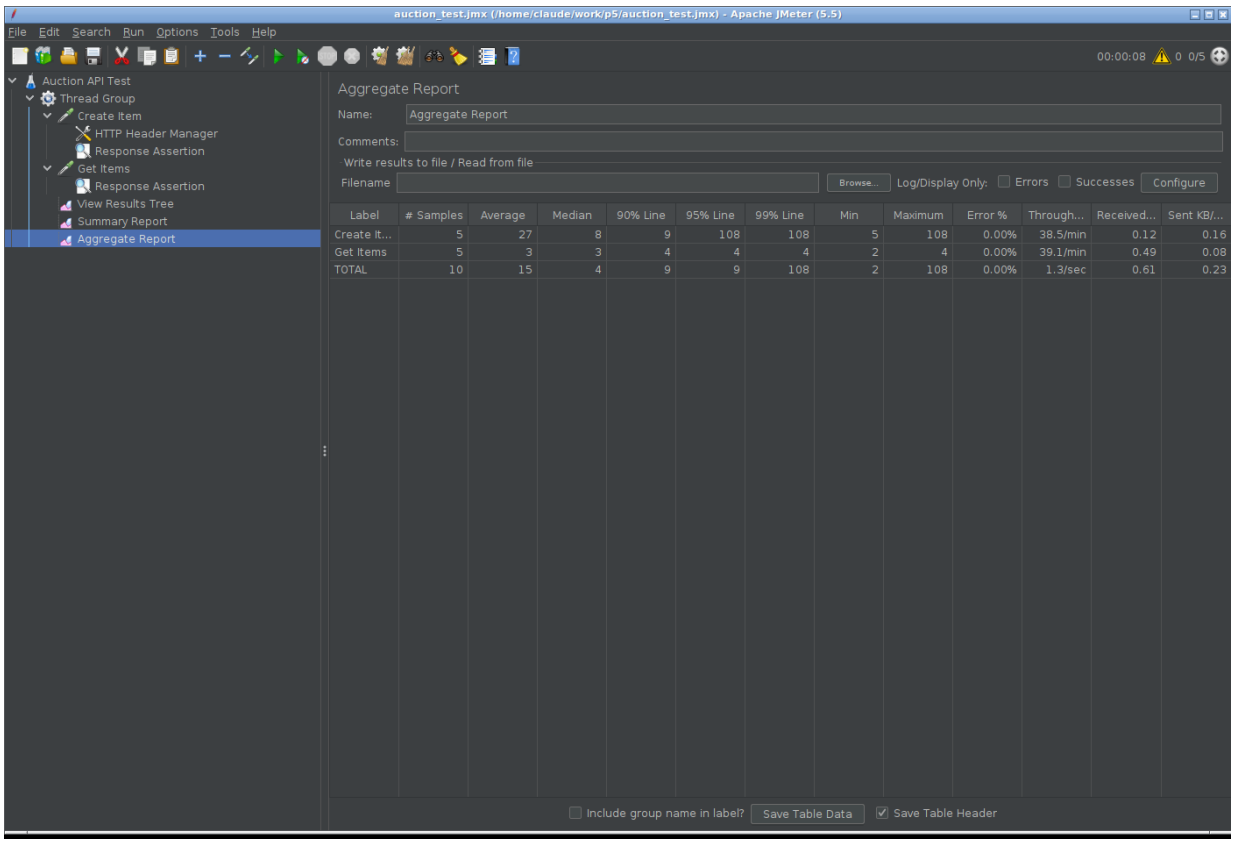



Рисунок 12 – Aggregate Report после выполнения теста

Таблица 2 – Результаты GUI-прогона (Summary Report / Aggregate Report)

| Запрос | Сэмплов | Среднее, мс | Медиана | 90 % | 95 % | 99 % | Min | Max | Ошибки | Пропускная способность |
|---|---|---|---|---|---|---|---|---|---|---|
| Create Item | 5 | 27 | 8 | 9 | 108 | 108 | 5 | 108 | 0,00 % | 38,5/мин |
| Get Items | 5 | 3 | 3 | 4 | 4 | 4 | 2 | 4 | 0,00 % | 39,1/мин |
| TOTAL | 10 | 15 | 4 | 9 | 9 | 108 | 2 | 108 | 0,00 % | 1,3/с |

### 6.2 Запуск в режиме командной строки

Тот же план выполнен в режиме без GUI с записью результатов в `results.jtl` и генерацией HTML-отчета: `jmeter -n -t auction_test.jmx -l out/results.jtl -e -o out/report`. Консольный вывод (рисунок 13) содержит итоговую строку `summary = 10 in 00:00:08 = 1.2/s Avg: 8 Min: 2 Max: 54 Err: 0 (0.00%)`. По файлу `.jtl` времена «Create Item» составили 54, 7, 6, 5, 5 мс (среднее 15,4), «Get Items» – 2, 4, 2, 2, 2 мс (среднее 2,4). HTML-

дашборд (рисунок 14) показывает APDEX 1,000 для обоих запросов и 0 % ошибок.

```
jmeter -n -t auction_test.jmx -l results.jtl -e -o report

$ jmeter/bin/jmeter -n -t auction_test.jmx -l out/results.jtl -e -o out/report
Creating summariser <summary>
Created the tree successfully using auction_test.jmx
Starting standalone test @ 2026 Sep 12 10:15:56 UTC (1789208156477)
Waiting for possible Shutdown/StopTestNow/HeapDump/ThreadDump message on port 4445
summary +      5 in 00:00:04 =    1.2/s Avg:    14 Min:     2 Max:    54 Err:     0 (0.00%) Active: 1 Started:
 3 Finished: 2
summary +      5 in 00:00:04 =    1.2/s Avg:     3 Min:     2 Max:     5 Err:     0 (0.00%) Active: 0 Started:
 5 Finished: 5
summary =     10 in 00:00:08 =    1.2/s Avg:     8 Min:     2 Max:    54 Err:     0 (0.00%)
Tidying up ...    @ 2026 Sep 12 10:16:04 UTC (1789208164717)
... end of run
```

Рисунок 13 – Консольный вывод non-GUI прогона JMeter

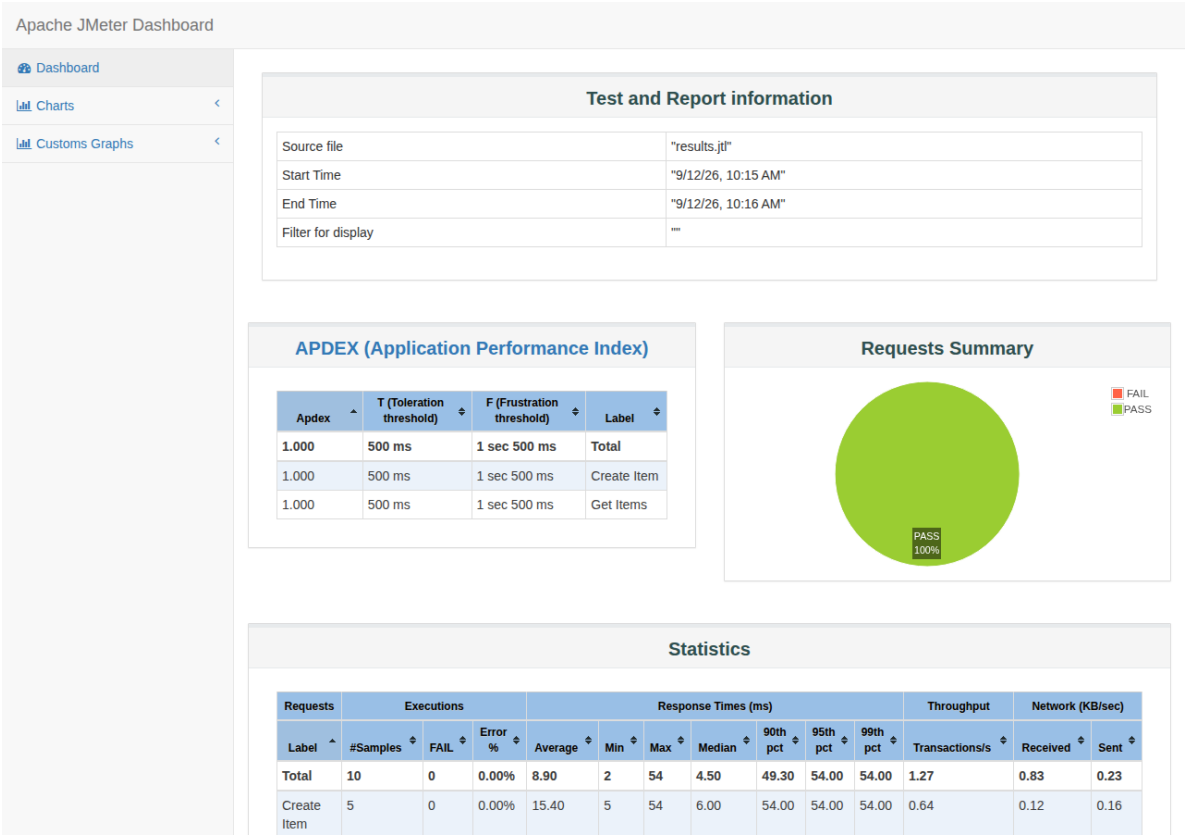



Рисунок 14 – HTML-отчет JMeter: APDEX и статистика запросов

Состояние базы данных после всех прогонов проверено запросом к `auction.db` (рисунок 15): 16 записей `Item 1` – одна от проверки curl и по пять от трех прогонов JMeter (контрольного, графического и консольного), что совпадает с 16 запросами `POST /items/` в журнале uvicorn.

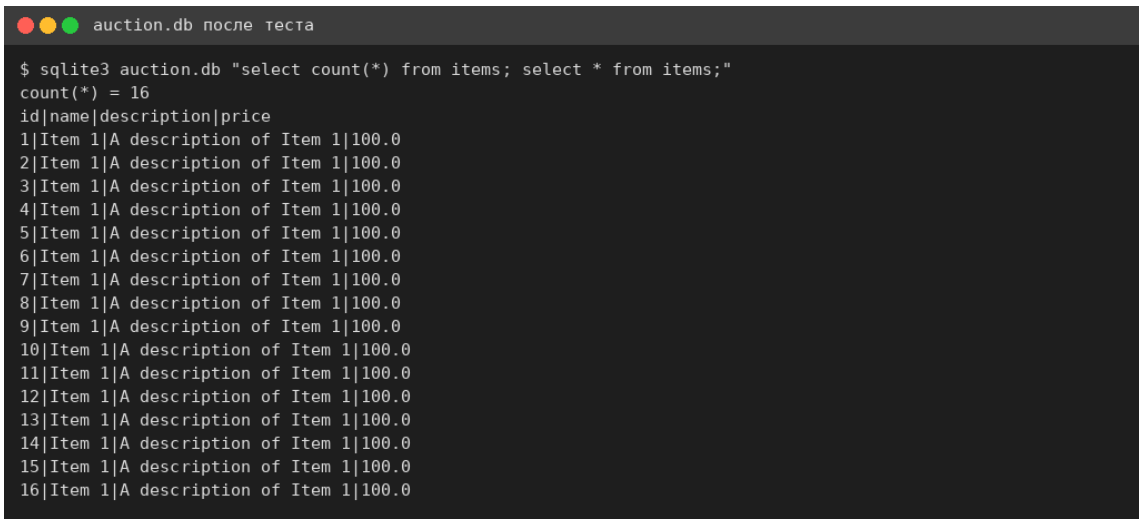



Рисунок 15 – Содержимое таблицы items после тестов

### 6.3 Анализ результатов

Сервер выдержал заданную нагрузку без ошибок; все ответы уложились в 108 мс, а медианные значения – в единицы миллисекунд. Первый запрос каждого прогона (54–108 мс) заметно медленнее последующих (2–9 мс): это прогрев HTTP-клиента JMeter и установка первого соединения, а не свойство сервера. Следует учитывать, что при пяти потоках и разгоне 10 с потоки стартуют раз в две секунды, поэтому нагрузка фактически последовательная и одновременных запросов не возникало; для настоящего нагрузочного испытания требуется увеличить число потоков и итераций и ввести таймеры. Замечено также, что размер ответа `GET /items/` растет с каждым POST (в JTL 271 → 559 байт за один прогон): в API нет пагинации, и при длительной нагрузке время ответа будет расти.

## 7 Найденные дефекты сервера

Таблица 3 – Дефекты тестируемого сервера

| № | Дефект | Проявление | Рекомендация |
|---|---|---|---|
| 1 | Поле id отсутствует в модели Item, но передается в конструктор | GET /items/ и GET /items/{id} возвращают объекты без id; записи неразличимы | Добавить `id: Optional[int] = None` или отдельную модель ItemOut |
| 2 | POST /items/ возвращает входной объект | Клиент не узнает id созданной записи | Возвращать созданную запись с `cursor.lastrowid` |

| № | Дефект | Проявление | Рекомендация |
|---|---|---|---|
| 3 | Аннотация `description: str = None` | В схеме OpenAPI поле выглядит обязательной строкой | Использовать `Optional[str] = None` |
| 4 | Нет пагинации в GET /items/ | Ответ растет с каждым POST (271 → 559 байт за прогон) | Добавить параметры `limit/offset` |
| 5 | Новое соединение SQLite на каждый запрос, файл БД в каталоге с `--reload` | На данной нагрузке ошибок нет; при росте нагрузки возможны блокировки | Пул соединений или отдельный каталог данных |

## 8 Выводы

В работе развернут REST-сервис на FastAPI и создан тестовый план Apache JMeter с группой из пяти виртуальных пользователей, двумя HTTP-запросами, менеджером заголовков, утверждениями и тремя слушателями. Тест выполнен в графическом и консольном режимах: все 10 запросов завершились кодом 200 без ошибок, среднее время отклика составило 15 мс (GUI) и 8 мс (CLI), максимальное – 108 мс на первом запросе; сформирован HTML-отчет с APDEX 1,000.

Помимо нагрузочных метрик, тестирование выявило функциональные дефекты сервера – отсутствие идентификаторов в ответах API и пагинации, – что показывает ценность даже простого нагрузочного плана как средства функциональной проверки. Освоены основные компоненты JMeter (Thread Group, Samplers, Config Elements, Assertions, Listeners) и порядок анализа результатов.